Physics-Constrained Machine Learning

Concepts, Tools, and Algorithms

Victor M. Zavala and Angan Mukherjee

Department of Chemical and Biological Engineering
University of Wisconsin-Madison

victor.zavala@wisc.edu

Module 4: Implementation and Case Studies



From Concepts to Practice

- **Module 1:** NNs, neural ODEs, and GPs are flexible, powerful tools to model complex systems from data; conceptually different (parametric vs. nonparametric).
- **Module 2:** physics can be embedded in data, model structure, loss, constraints, and training space; physics regularizes ML models and narrows down the training space.
- **Module 3:** params are estimated by MLE or MAP (prior as regularizer); uncertainty is quantified by Laplace, VI, or (for GPs) directly, and propagated through functions.
- **Module 4:** implementing these ideas (tools) and applying them:
  - Ionic liquids (property prediction)
  - Microbial communities (dynamic modeling)
  - Physics-informed Bayesian optimization (process design and operation)
  - Solvent design (experimental design)

Implementation Workflow

1. **Define the problem:** inputs, outputs, available data, and available physics (balances, thermodynamics, kinetics, approximate models).
2. **Choose an ML architecture** (Module 1): NN (static), neural ODE (dynamic), or GP (small data, built-in UQ).
3. **Choose how to fuse physics** (Module 2): features, additive, physics-informed (PINN), physics-constrained (PCNN, UDE), or filtering.
4. **Estimate params and quantify uncertainty** (Module 3): MAP with priors; Laplace, VI, or GP uncertainty; propagation through functions.
5. **Use the model in decisions:** Bayesian optimization, experimental design, real-time optimization, and control.

- Each step has mature, open-source tools.

Tools: Machine Learning and Differentiable Programming

- **Automatic differentiation and training** (back propagation, Adam; Modules 1-3):
  - PyTorch, JAX, TensorFlow (Python); Flux (Julia)
  - scikit-learn for classical ML models (e.g., small NNs, GPs)

- **Neural ODEs and UDEs** (repeated simulation and fitting of trajectories):
  - torchdiffeq (PyTorch), Diffrax (JAX), SciML/DiffEqFlux (Julia)
  - Simple fixed-step solvers written directly in PyTorch or JAX (as in our notebooks)

- **Physics-informed libraries:**
  - DeepXDE (PINNs, operator learning), NeuroMANCER (constrained learning, control)

- Workshop notebooks use PyTorch, NumPy, SciPy, and scikit-learn.

Tools: Gaussian Processes, UQ, and Bayesian Optimization

- **Gaussian processes** (Modules 1 and 3):
  - scikit-learn (simple, small data), GPyTorch and GPflow (scalable, GPU)
- **Bayesian inference and UQ** (Module 3):
  - Automatic differentiation tools give Hessians (Laplace) and gradients (VI)
  - Pyro and NumPyro, TensorFlow Probability (VI); PyMC and Stan (sampling)
- **Bayesian optimization** (Module 4):
  - BoTorch and Ax (PyTorch); custom implementations with scikit-learn and SciPy

Tools: Optimization, Constraints, and Domain Physics

- **Constrained optimization** (hard constraints, simultaneous training; Modules 2 and 3):
  - Pyomo, JuMP, CasADi with nonlinear solvers such as Ipopt
  - OMLT: embed trained NNs and GPs in optimization models
- **Domain physics** (physics components, features, and filters):
  - Process simulators (e.g., Aspen), thermodynamic models (e.g., COSMO-RS)
  - Molecular features (e.g., RDKit), electronic structure (e.g., DFT)
  - Property databases (e.g., NIST ILThermo)
- Physics must be differentiable for PINNs, PCNNs, UDEs (write it in the ML framework), but can be a black-box simulator for additive models and filtering.

Tools Are Easy to Access

- Nearly all tools are open-source, free, and installed with one command (`pip` or `conda`).
- Free cloud computing (e.g., Google Colab) runs notebooks in a browser, no installation.
- Small and medium physics-informed models train in seconds to minutes on a laptop; all workshop notebooks run in a few minutes.
- Code of many research papers is publicly available (e.g., GitHub), which facilitates reproducing and extending results.
- The barrier is no longer software; it is understanding the concepts (Modules 1-3).

Large Language Models Facilitate Implementation

- Large language models (LLMs) and coding assistants make implementation fast:
  - Translate equations (e.g., balances, ELBO) into code
  - Generate training loops, solvers, and plots; explain unfamiliar libraries
  - Debug errors and refactor code to follow a desired notation
- Workshop notebooks were developed with the help of an LLM coding assistant.
- However, the user must provide the concepts and verify the results:
  - Check physics (units, balances, signs) and numbers (e.g., against known solutions)
  - Check that models are trained and uncertainty is quantified correctly
- Concepts (Modules 1-3) + tools + LLMs make physics-constrained ML accessible.

Workshop Notebooks

Notebook	Concepts	Tools
`PCML_Module1_NN_NODE_CSTR`	NN, neural ODE, Adam	PyTorch, SciPy
`PCML_Module1_GP_CSTR`	GP, kernels, MLE of hyperparams	scikit-learn
`PCML_Module2_Membrane`	Additive GP, PINN, PCNN, mass balances	PyTorch, scikit-learn
`PCML_Module2_UDE_CSTR`	UDE vs. neural ODE, filtering	PyTorch, SciPy, scikit-learn
`PCML_Module3_Spurious_HougenWatson`	Spurious solutions, bounds on params	SciPy
`PCML_Module3_Bayes_LinearModel`	Prior, likelihood, posterior	NumPy
`PCML_Module3_UQ_Membrane`	MLE, MAP, Laplace, VI, propagation	PyTorch, scikit-learn
`PEQ_Colloquium_Tutorial_1`	Hands-on: PINN, PCNN (Ipopt), bootstrap	PyTorch, Pyomo, Ipopt
`PEQ_Colloquium_Tutorial_2`	Hands-on: neural ODE, ensembles, VI	JAX

- All notebooks follow the notation of the slides and run on a laptop or Google Colab; PEQ tutorials are given at the end of the workshop.

Case Studies: Overview

Case study	**ML (Module 1)**	**Physics (Module 2)**	**UQ (Module 3)**
Ionic liquids	NN	Features, additive	Generalization
Microbial communities	Neural ODE	UDE (how much physics?)	MAP, VI
Physics-informed BO	GP	Additive, filtering, composite	GP UQ, propagation
Solvent design	GP	Prioritization (COSMO-RS)	GP UQ

- Each case study combines concepts of Modules 1-3 using the tools above.

Case Study: Ionic Liquids for Batteries

- **Goal:** predict ionic conductivity of ionic liquids (ILs) for safe battery electrolytes; about 100 descriptors for 218 ILs.
- **Concepts:** NN (Module 1); physics-informed features and additive hybrid NE model + NN (Module 2); NN params estimated by MLE; generalization limits (Module 3).
- **Tools:** experimental property databases (NIST ILThermo), molecular descriptors (RDKit), and molecular graphs.
- **Lessons:** physical descriptors reduce error relative to NE; molecular descriptors alone miss collective (long-range) effects governing IL conductivity.

Umaña, J. E., Cashen, R. K., Zavala, V. M., and Gebbie, M. A. (2025). Uncovering ion transport mechanisms in ionic liquids using data science. Digital Discovery, 4(6), 1423-1436. doi:10.1039/d4dd00378k

Case Study: Ionic Liquids for Batteries

$\sim$ 100 Descriptors

ILThermo
Conductivity
Viscosity
Density

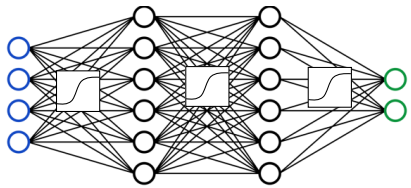
RDKit Descriptors
Structure
Geometry
Polarity

Graphs
2D Structure
Connectivity

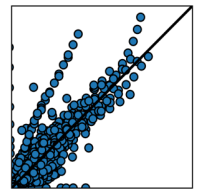
$\mathbf{x}$ $\Longrightarrow$

$$\Lambda_{NE} = \frac{N_A e^2}{6\pi\mu}\left(\frac{1}{R_+} + \frac{1}{R_-}\right)$$

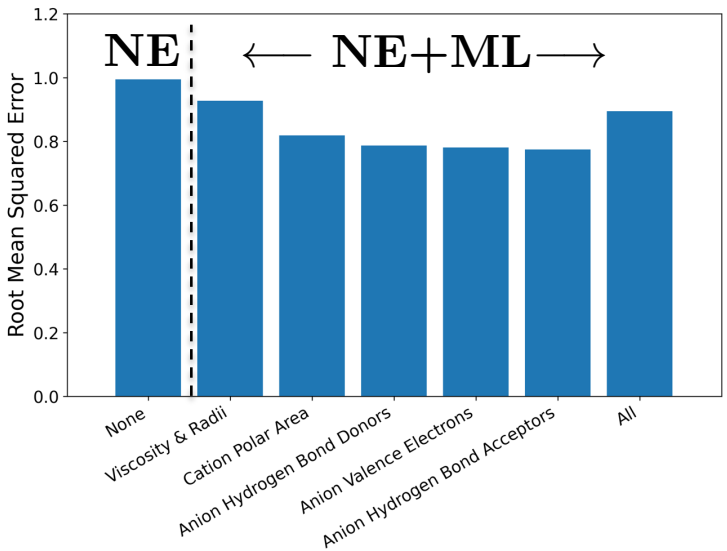
$$+$$

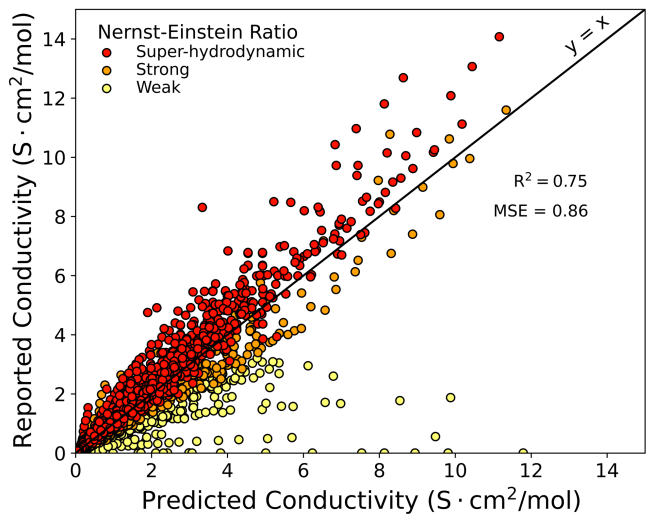


$\mathrm{Ph}(\mathbf{x},\theta) + \mathrm{NN}(\mathbf{x},\theta)$ $\Longrightarrow$



Conductivity





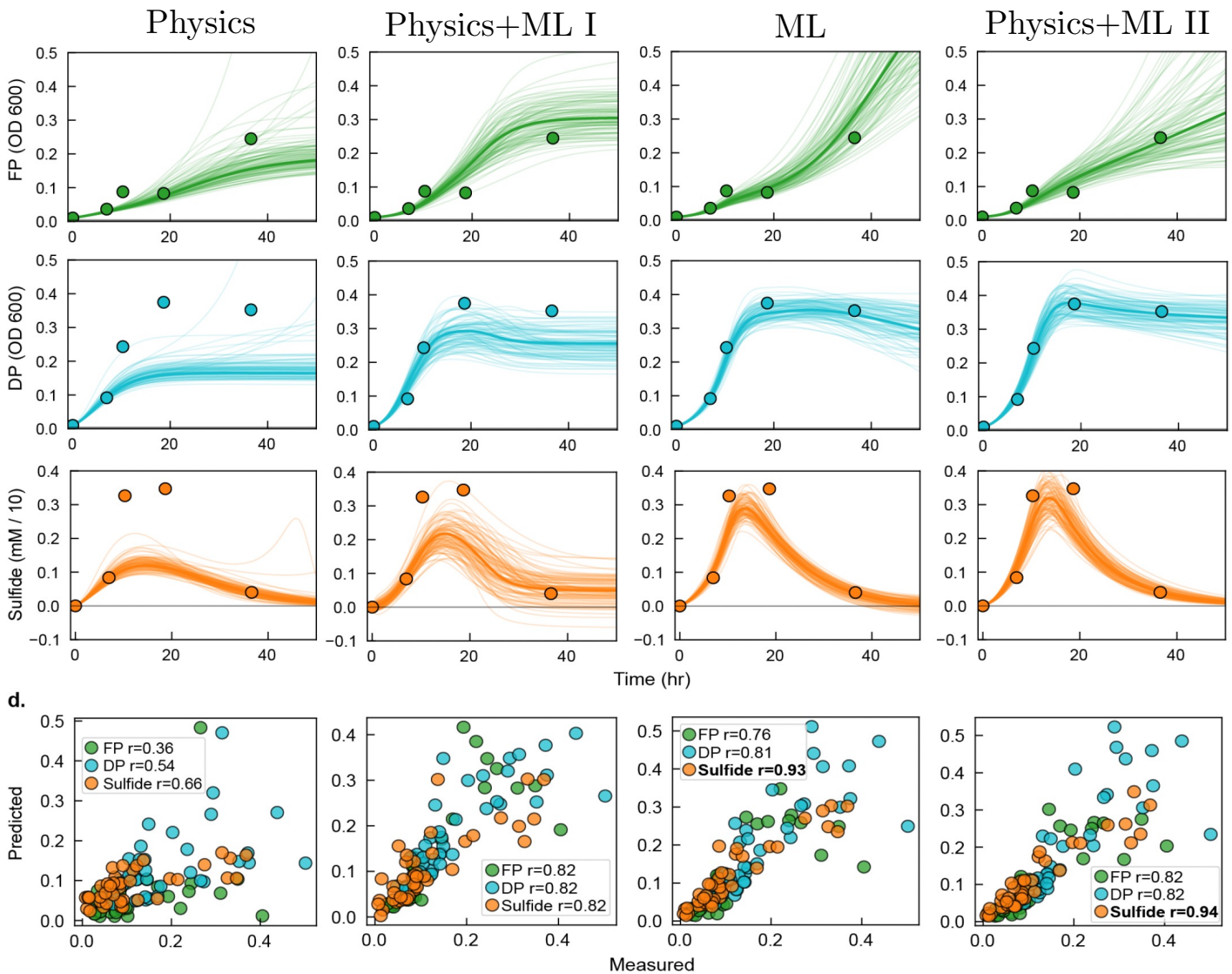
Umaña, J. E., Cashen, R. K., Zavala, V. M., and Gebbie, M. A. (2025). Uncovering ion transport mechanisms in ionic liquids using data science. Digital Discovery, 4(6), 1423-1436. doi:10.1039/d4dd00378k

Case Study: Microbial Communities

- **Goal:** predict species and metabolite dynamics in microbial communities from scarce data.
- **Concepts:** neural ODEs (Module 1); UDEs with consumer-resource structure, spectrum from physics to ML (Module 2); MAP with Gaussian priors, VI for UQ, hyperparams by expectation maximization (Module 3).
- **Tools:** VI solved with Adam; code on GitHub.
- **Lessons:** hybrid models give the most consistent predictions; UQ provides insight into generalization; how to fuse physics and ML is not obvious.

Thompson, J., Connors, B. M., Zavala, V. M., and Venturelli, O. S. (2026). Physics-constrained neural ordinary differential equation models to discover and predict microbial community dynamics. PNAS, 123(13), e2517661123. doi:10.1073/pnas.2517661123

Case Study: Microbial Communities



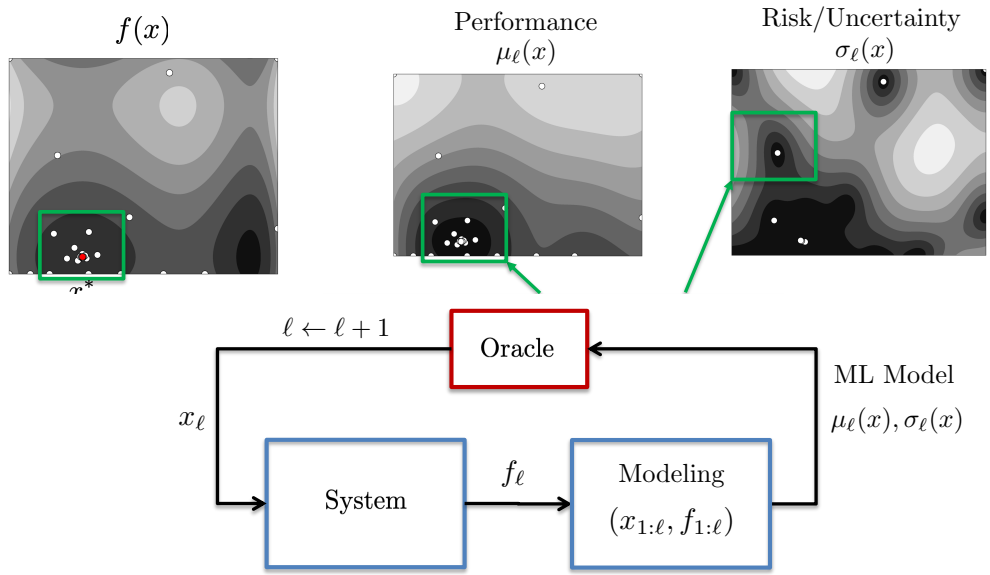
Thompson, J., Connors, B. M., Zavala, V. M., and Venturelli, O. S. (2026). Physics-constrained neural ordinary differential equation models to discover and predict microbial community dynamics. PNAS, 123(13), e2517661123. doi:10.1073/pnas.2517661123

Case Study: Bayesian Optimization (BO)

- **Goal:** find inputs x that optimize performance $f(x)$ of an expensive system (experiments or simulations) with few evaluations.
- BO builds a GP of performance (Module 1) and uses its uncertainty (Module 3) to select next input with an acquisition function:

$$x_{\ell+1} \in \underset{x \in X}{\operatorname{argmax}} \ \mu_\ell(x) + \kappa \cdot \sigma_\ell(x)$$

$\mu_\ell(x)$ rewards performance (exploitation) and $\sigma_\ell(x)$ rewards information (exploration).

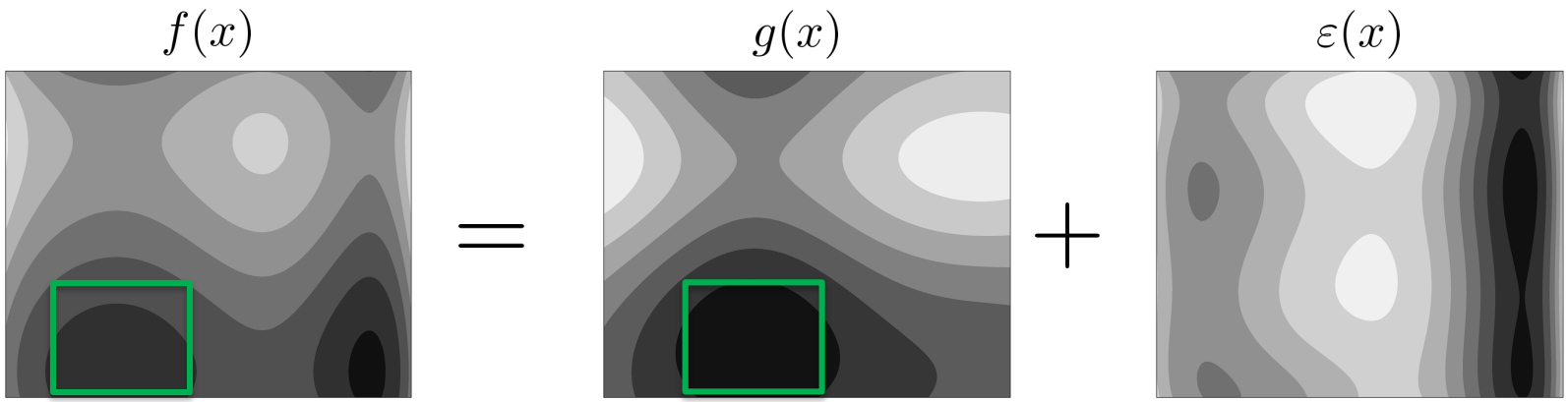


Paulson, J. A., and Tsay, C. (2025). Bayesian optimization as a flexible and efficient design framework for sustainable process systems. Curr. Opin. Green Sustain. Chem., 51, 100983. doi:10.1016/j.cogsc.2024.100983

Physics-Informed BO: Reference Model

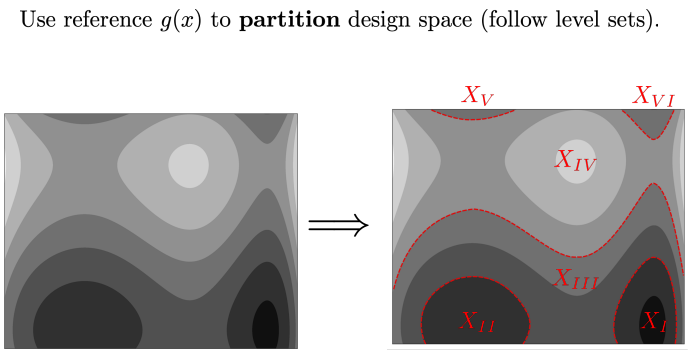
- Standard BO treats system as a black box; we often have prior knowledge (e.g., physics, low-resolution model).
- Decompose performance as $f(x) = g(x) + \epsilon(x)$: reference model $g(x) = \mathrm{PH}(x)$ and GP residual $\epsilon(x) = \mathrm{ML}(x)$.
- This is the *additive* structure of Module 2 (GP with physics as prior mean); reference model captures coarse features and residual focuses the search.

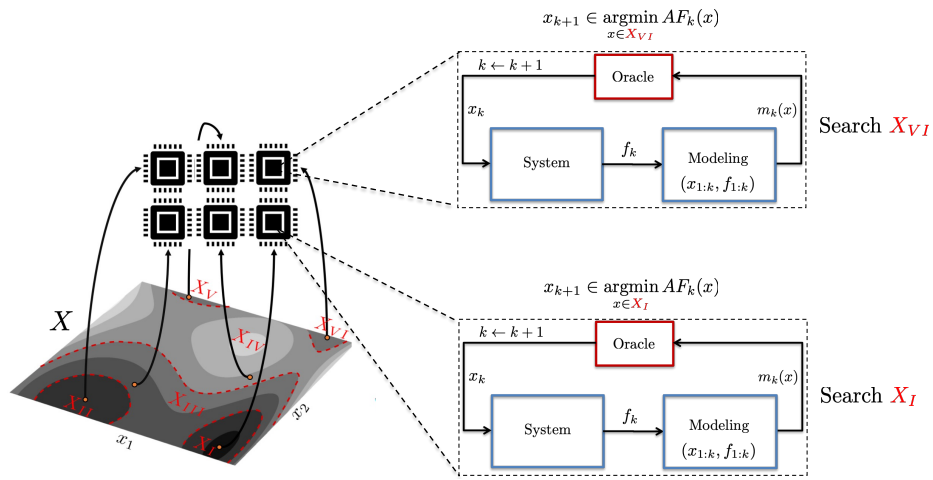
$$\text{Decompose performance as } f(x) = g(x) + \epsilon(x) \quad \begin{cases} g(x) \text{ is reference model (knowledge)} \\ \epsilon(x) \text{ is residual model (unknown)} \end{cases}$$



Physics-Informed BO: Domain Partitioning

- Use reference model $g(x)$ to partition the design space along its level sets; each partition is searched by an independent BO agent.
- This is physics-informed *filtering* (Module 2): physics narrows down where to sample.
- Enables parallel, high-throughput experiments; no communication between BO agents.





González, L. D., and Zavala, V. M. (2023). New paradigms for exploiting parallel experiments in Bayesian optimization. Comput. Chem. Eng., 170, 108110. doi:10.1016/j.compchemeng.2022.108110

Example: Parallel BO for Reactor System

- Two reactors with temperatures T_1, T_2; minimize operating cost; simplified physics gives reference $g(x)$ that partitions the space.
- Level-set partitions and parallel experiments accelerate search vs. standard BO.
- **Tools:** Python, GPs from scikit-learn (Matérn kernel), acquisition optimized with SciPy (L-BFGS-B); code on GitHub (zavalab/bayesianopt).

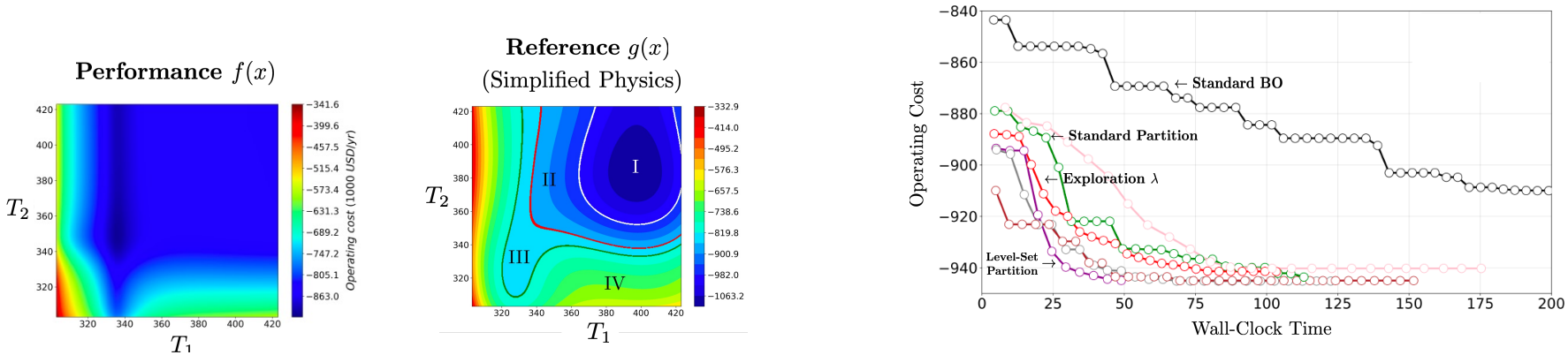


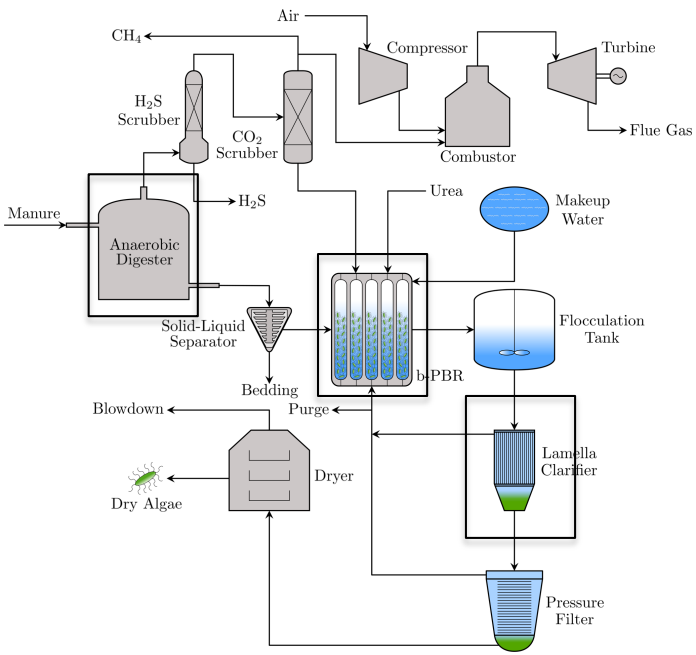
Figure: Performance and reference model (left); operating cost vs. wall-clock time (right).

Physics-Informed BO: Composite Systems

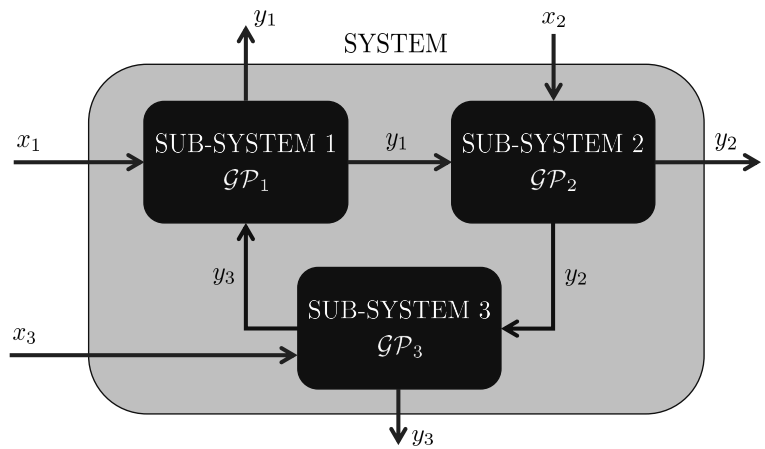
- Performance is often a known function of unknown outputs of connected subsystems:

$$f(x, y(x)), \qquad y(x) \sim \mathrm{GP}(x)$$

- *Composite* hybrid $\mathrm{PH}(x, \mathrm{ML}(x))$ (Module 2): GPs model subsystems, physics links them.
- **Key:** propagate uncertainty of y to f (Module 3): sampling or linearization (BOIS).



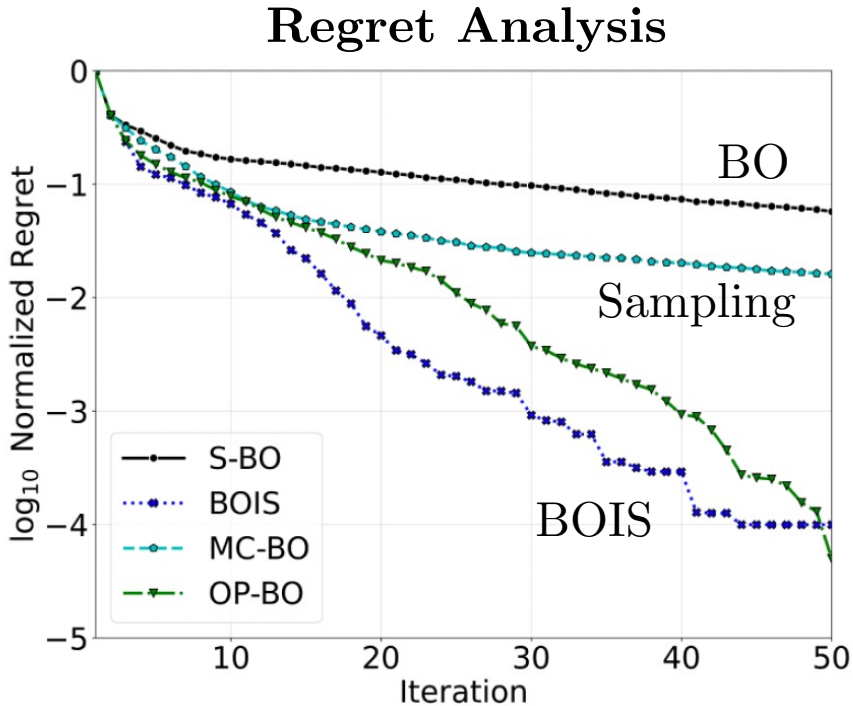




Paulson, J. A., and Lu, C. (2022). COBALT: Constrained Bayesian optimization of computationally expensive grey-box models exploiting derivative information. Comput. Chem. Eng., 160, 107700. doi:10.1016/j.compchemeng.2022.107700

Example: BOIS for Process Optimization

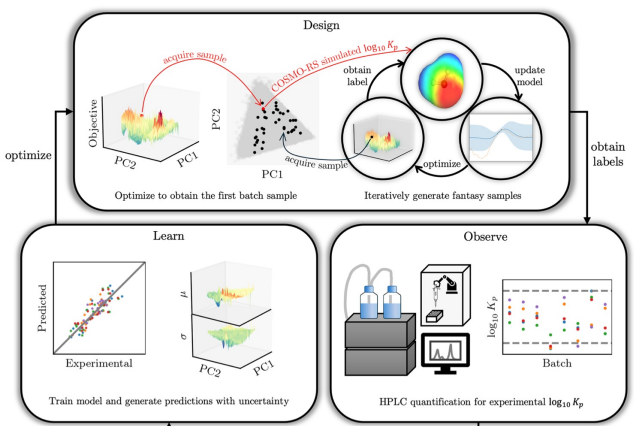
- Nutrient recovery process (anaerobic digester and photobioreactor); GPs model units and the process model connects them.
- BOIS uses adaptive linearization (Module 3) to obtain closed-form mean and variance of f; it converges much faster than standard BO and than sampling-based BO.
- **Tools:** Python, GPs from scikit-learn, acquisition optimized with SciPy (SLSQP, multi-start), gradients by finite differences; code on GitHub (zavalab/bayesianopt).
- **Key:** physics greatly accelerates search; linearization is more efficient than sampling.

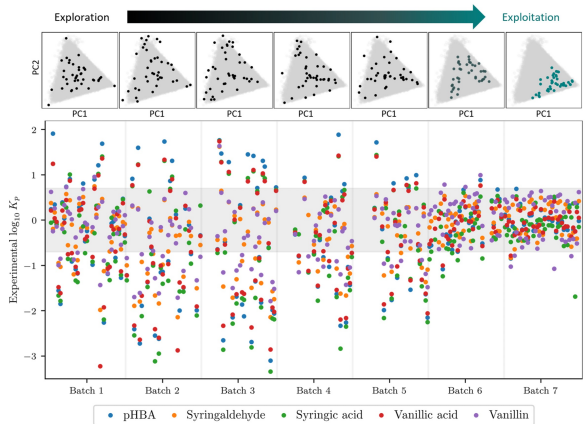


González, L. D., and Zavala, V. M. (2025). Implementation of a Bayesian optimization framework for interconnected systems. Ind. Eng. Chem. Res., 64(4), 2168-2182. doi:10.1021/acs.iecr.4c02915

Case Study: Solvent Design for Separations

- **Goal:** identify green solvent mixtures to separate lignin-derived products; predict partition coefficients $\log K_p$ over a combinatorial space of mixtures.
- **Concepts:** GPs for $\log K_p$ (Module 1); physics model (COSMO-RS) prioritizes mixtures and provides "fantasy" labels to select batches of experiments (Module 2); GP hyperparams by MLE; GP uncertainty guides exploration and exploitation (Module 3).
- **Tools:** GPs from scikit-learn (Matérn kernel), COSMO-RS (COSMOtherm), DFT (TURBOMOLE), PCA to visualize design space.
- **Lessons:** new green solvent mixtures found within seven batches; model error lower than COSMO-RS alone and decreasing with each batch.





Qin, S., et al. (2024). Identifying green solvent mixtures for bioproduct separation using Bayesian experimental design. ACS Sustain. Chem. Eng., 12(52), 18634-18647. doi:10.1021/acssuschemeng.4c07423

Hands-On Tutorials

- Two tutorials reinforce Modules 1-3; relevant notebooks provided.
- **Tutorial 1** (`PEQ_Colloquium_Tutorial_1`): membrane; NN, PINN, and PCNN with hard constraints (Ipopt); UQ by constrained bootstrap.
- **Tutorial 2** (`PEQ_Colloquium_Tutorial_2`): CO oxidation microkinetics; neural ODE, ensemble of neural ODEs, and VI for a physics-constrained (microkinetic) model.
- Each tutorial includes a table that maps its notation to the notation of the slides.

Tutorial 1: Constrained NNs for a Membrane Separator

- Membrane with pressure as input and four flows as outputs; NN with two neurons (16 params); mass balances $F_p + F_r = F$, $S_p + S_r = Fc_f$.
- NN (Module 1), PINN with physics loss (Module 2), and PCNN with balances imposed as linear constraints on the NN weights and solved with Pyomo and Ipopt (simultaneous approach; Module 2 and constraints on parameters in Module 3).
- Test RMSE of F_p: 1.05 (NN), 0.44 (PINN), and 0.41 kg/h (PCNN); maximum mass-balance violation: 2.9, 0.14, and 10^{-14} kg/h.

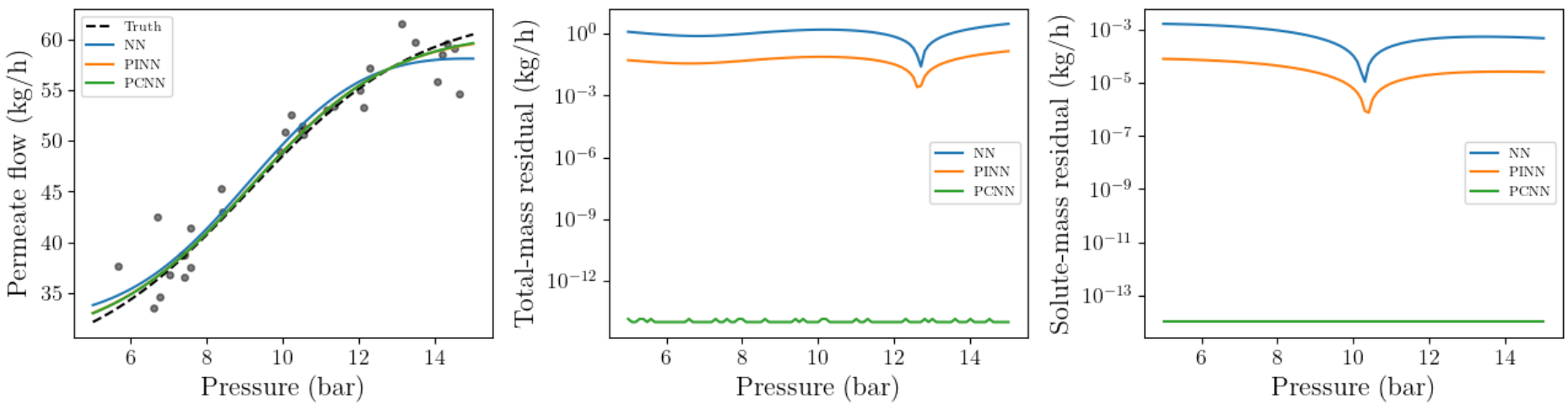


Figure: Permeate flow predictions (left) and balance residuals (middle and right) for NN, PINN, and PCNN.

Tutorial 1: Uncertainty Quantification by Bootstrapping

- Constrained parametric bootstrap (Module 3): generate data from fitted PCNN plus noise, refit with Ipopt (30 refits, 23 s), propagate params to predictions.
- Every bootstrap prediction satisfies the mass balances exactly (residuals below 10^{-13} kg/h): physically consistent uncertainty bands.
- Bootstrapping is simple but refits repeatedly; VI and Laplace avoid refitting (Module 3).

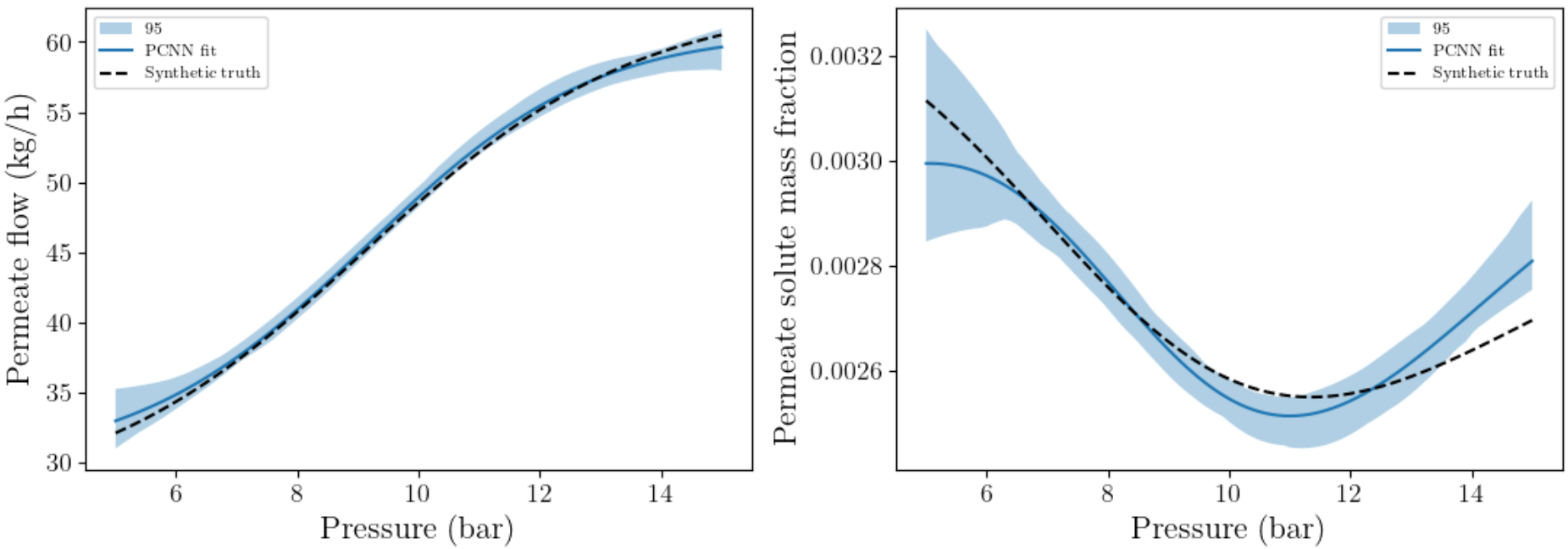


Figure: PCNN predictions with 95% bootstrap bands for permeate flow and permeate solute fraction.

Tutorial 2: Neural ODEs and VI for Microkinetic Modeling

- CO oxidation on a surface: four coverages $(\phi_{CO}, \phi_O, \phi_{CO_2}, \phi_S)$ governed by a microkinetic ODE with four unknown rate constants k.

- Three models are compared (implemented in JAX):
  - Single neural ODE (Module 1): 148 params, trained by repeated simulation; no UQ.
  - Ensemble of 50 neural ODEs (Module 3): spread reflects initialization, not a posterior.
  - VI for physics-constrained microkinetic model (Modules 2-3): params $\theta = \log k$, Gaussian prior; mean-field $q(\theta|\lambda)$; ELBO maximized with Adam.

Tutorial 2: Neural ODEs and VI for Microkinetic Modeling

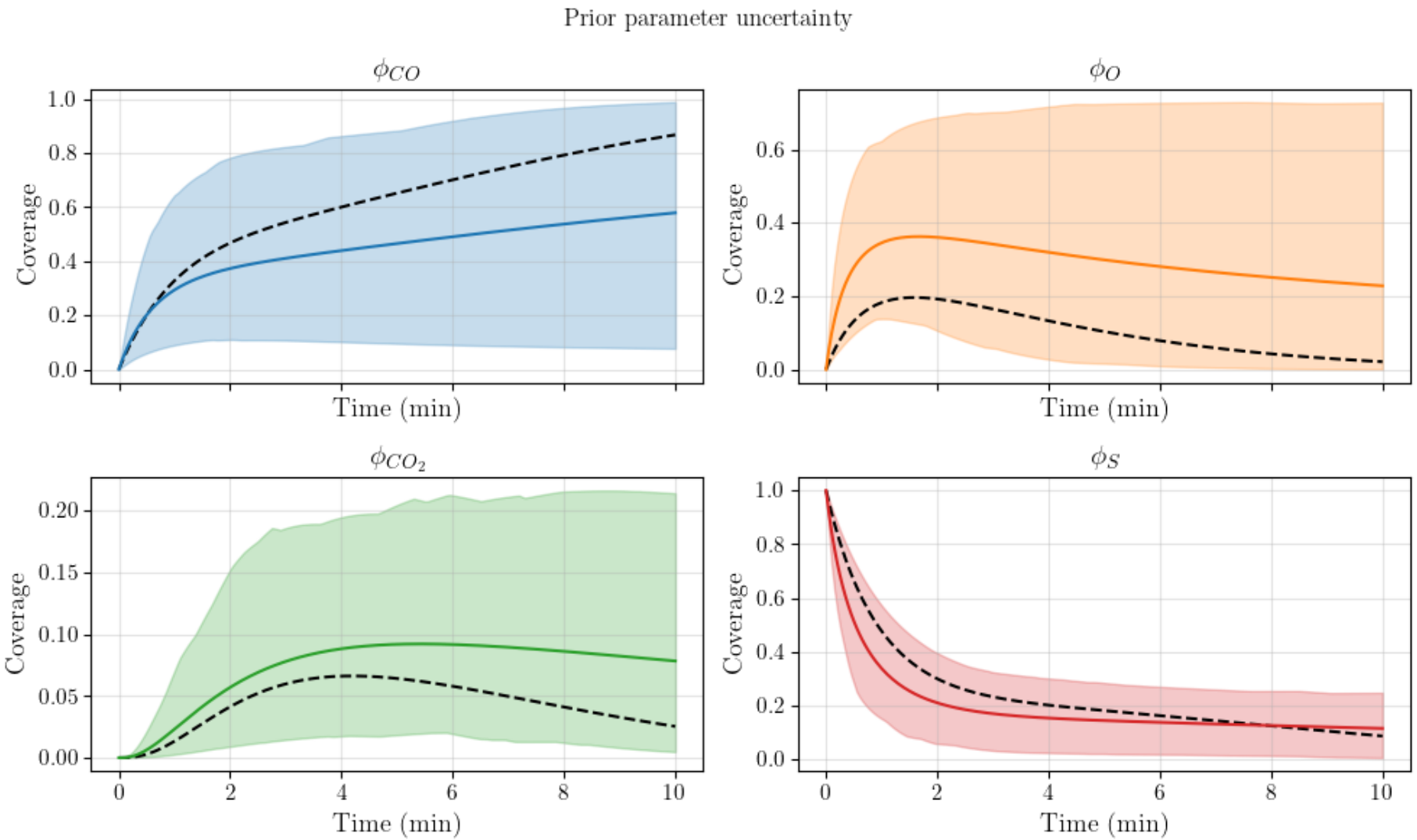


Figure: Prior predictive uncertainty of microkinetic model (before data).

Tutorial 2: Comparison of Models

- All three models fit the data; only VI provides uncertainty of a physically consistent model (rate constants with posterior intervals).
- VI estimates params and their uncertainty simultaneously with a single optimization (about 30 s); sampling the posterior then propagates uncertainty to predictions.

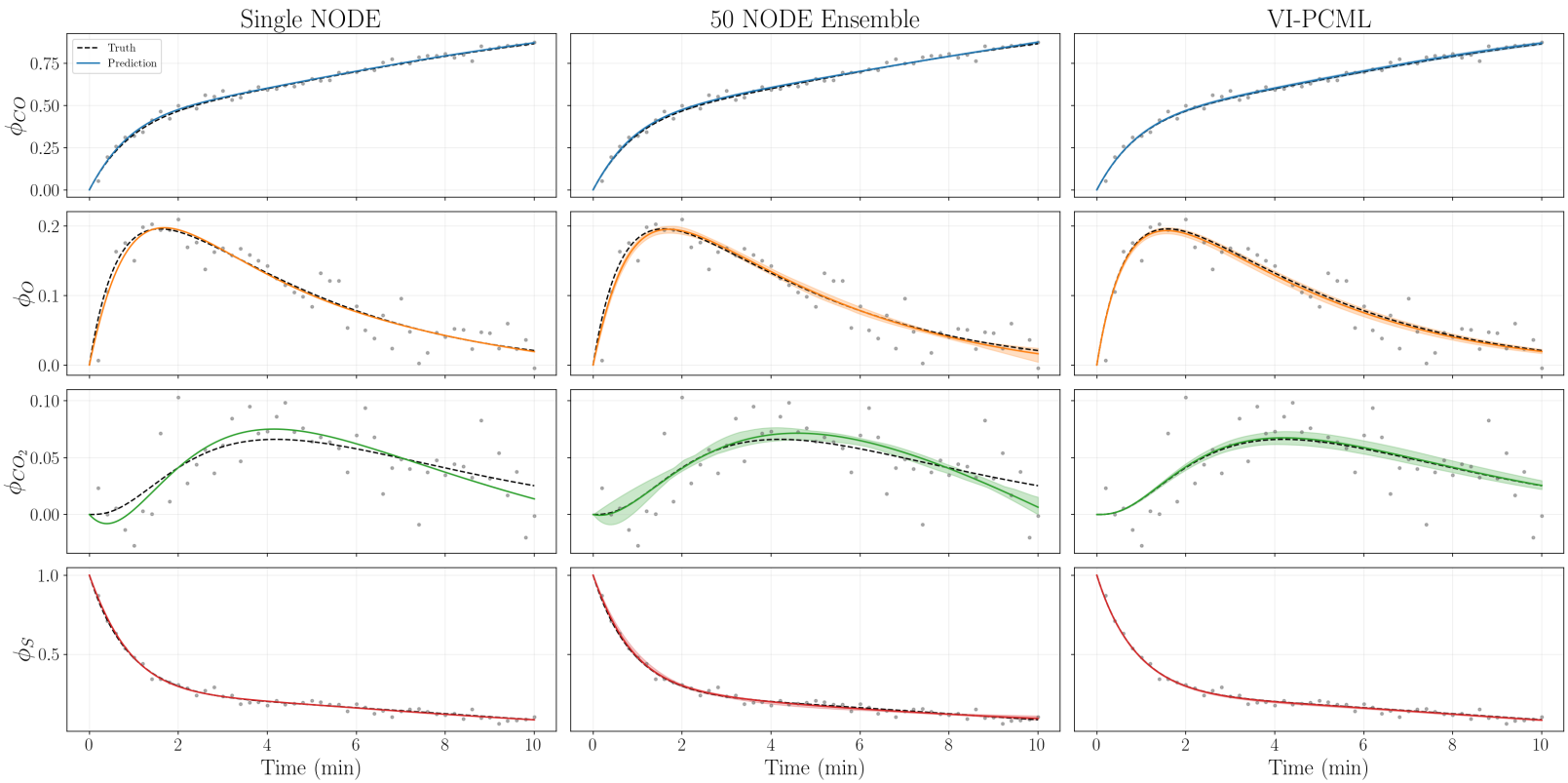


Figure: Single neural ODE (left), 50-model ensemble (middle), and VI for microkinetic model (right): truth (dashed), predictions, and data.

References

- Mukherjee, A., and Zavala, V. M. (2026). Physics-constrained machine learning for chemical engineering. Curr. Opin. Chem. Eng., 51, 101228. doi:10.1016/j.coche.2026.101228
- Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. NeurIPS. doi:10.48550/arXiv.1912.01703
- Wächter, A., and Biegler, L. T. (2006). On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. Math. Program., 106(1), 25-57. doi:10.1007/s10107-004-0559-y
- Umaña, J. E., Cashen, R. K., Zavala, V. M., and Gebbie, M. A. (2025). Uncovering ion transport mechanisms in ionic liquids using data science. Digital Discovery, 4(6), 1423-1436. doi:10.1039/d4dd00378k
- Thompson, J., Connors, B. M., Zavala, V. M., and Venturelli, O. S. (2026). Physics-constrained neural ordinary differential equation models to discover and predict microbial community dynamics. PNAS, 123(13), e2517661123. doi:10.1073/pnas.2517661123
- González, L. D., and Zavala, V. M. (2023). New paradigms for exploiting parallel experiments in Bayesian optimization. Comput. Chem. Eng., 170, 108110. doi:10.1016/j.compchemeng.2022.108110
- González, L. D., and Zavala, V. M. (2025). Implementation of a Bayesian optimization framework for interconnected systems. Ind. Eng. Chem. Res., 64(4), 2168-2182. doi:10.1021/acs.iecr.4c02915
- Qin, S., et al. (2024). Identifying green solvent mixtures for bioproduct separation using Bayesian experimental design. ACS Sustain. Chem. Eng., 12(52), 18634-18647. doi:10.1021/acssuschemeng.4c07423
- Paulson, J. A., and Lu, C. (2022). COBALT: Constrained Bayesian optimization of computationally expensive grey-box models exploiting derivative information. Comput. Chem. Eng., 160, 107700. doi:10.1016/j.compchemeng.2022.107700
- Paulson, J. A., and Tsay, C. (2025). Bayesian optimization as a flexible and efficient design framework for sustainable process systems. Curr. Opin. Green Sustain. Chem., 51, 100983. doi:10.1016/j.cogsc.2024.100983